

¿Qué es JSX en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

JSX significa Javascript XML, es una extensión de sintaxis de javascript que permite escribir código HTML dentro de archivos JS. Parece HTML pero tiene todas las funcionalidades de JS. Ese código JSX se compila a código JS para que los navegadores puedan entenderlo. Características:

- Cualquier expresión de JS se puede usar en el código HTML si se pone entre {}, que indica que es una expresión y no un objeto HTML.
- Para diferenciar palabras reservadas de JS y HTML (ya que comparten varias) algunas palabras de HTML cambian y se usa camelCase. Por ejemplo class pasa a ser className
- React evita los ataques XSS porque convierte a texto plano las expresiones {} nombradas.

JSX sirve para indicar cómo debe verse la interfaz de usuario de forma declarativa, lo cual es más natural que programarlo de forma imperativa.

Este es un ejemplo de código jsx usado para programar un componente "Tarjeta" de React. Se ve cómo definimos una función Tarjeta igual que en JS, dicha función devuelve el componente. Dentro de la función JS tenemos código HTML: usamos etiquetas div, header, h1, etc. También se ve que en los atributos se usa className en vez de class, por lo explicado. Entre corchetes están las expresiones JS, por ejemplo {item.titulo} en la etiqueta h1 indica que lo que queremos escribir es el resultado de evaluar una expresión.

```

import './Tarjeta.css'
import { useState } from 'react'

export function Tarjeta({item, appeals, manejoAppeals, manejoNext}) {

  const text = appeals ? 'Appealing!' : 'Appeals to me'
  const claseBoton = appeals ? 'appealing' : 'notAppealing'

  return (
    <div className="tarjeta">
      <header className="tarjeta-header">
        <h1 className="tarjeta-title">{item.titulo}</h1>

        {item.subtitulo && (
          <h2 className="tarjeta-autor">{item.subtitulo}</h2>
        )}

        <p className="tarjeta-categorie">{item.categoria}</p>
      </header>

      <div className="tarjeta-cuerpo">
        <img className="tarjeta-imagen" src={item.imagenUrl} alt="" />

        {item.descripcion && (
          <p> {item.descripcion} </p>
        )}
      </div>

      <div className="tarjeta-botones">
        <button className={`tarjeta-appeals ${claseBoton}`} onClick={manejoAppeals}>
          {text}
        </button>

        <button className="tarjeta-skip" onClick={manejoNext}>
          Next
        </button>
      </div>
    </div>
  )
}

```

¿Qué son los componentes en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

En React la interfaz gráfica está compuesta de piezas llamadas componentes. Esas piezas se programan de forma independiente, lo cual los hace reutilizables en el proyecto y también en otros proyectos.

Características:

Permiten programar la IU de forma modular y mediante la creación de código reutilizable.

Gracias a que la interfaz está hecha con pedazos independientes, React puede actualizar solo las partes de la pantalla que cambian (virtual DOM). Debido a la reutilización reducen el tiempo de desarrollo.

Para un ejemplo ilustrativo, considere el código presentado en el inciso anterior junto con:

```

function App() {
  const [indice, setIndice] = useState(0)

  const [likes, setLikes] = useState([])

  const elementoActual = ELEMENTOS[indice]

  const esAppealing = likes.includes(elementoActual.id)

  //funciones para manejar clicks
  const clickAppeals = () => {
    setLikes((prevLikes) =>
      esAppealing
      ? prevLikes.filter((id) => id !== elementoActual.id)
      : [...prevLikes, elementoActual.id]
    );
  }

  const clickNext = () => {
    setIndice((prev) => (prev+1) % ELEMENTOS.length)
  }

  return (
    <>
      <Tarjeta
        item={elementoActual}
        appeals={esAppealing}
        manejoAppeals={clickAppeals}
        manejoNext={clickNext}
      />
    </>
  )
}

```

La función App es el main del proyecto React. Vemos que devuelve (renderiza) el componente Tarjeta mostrado más arriba. En este caso no hay más componentes, pero esto sirve para ver que en el main tenemos las instanciaciones de nuestros componentes, ya que así se construye la IU.

¿Qué son los props en React? ¿Cuáles son sus características y para qué sirve?
Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

Prop es la abreviatura de propiedades. Las props son los argumentos que se le pasan a un componente. Vimos que los componentes se modelan como una función de JS cuyo valor

de retorno luego es renderizado (se dice que los componentes crean elementos, que son los que aparecen en la IU). Por lo tanto, los componentes reciben (o no) parámetros.

Características:

- Los componentes padre pueden pasarle sus props a sus componentes hijo
- Son de solo lectura, es decir que un hijo recibe un prop de su padre y esto solo le indica el valor de ese argumento, el hijo no puede hacer cambios visibles por su padre.
- Son un objeto de JS, si nosotros le pasamos muchos argumentos (props) a un componente, React en realidad los guarda todos en un único objeto de JS llamado prop.

Sirven para hacer reutilizables a los componentes (que se comporten de determinada forma según sus entradas, en vez de que esté ya definido) y para compartir información entre componentes padre y componentes hijo.

Para el ejemplo veamos nuevamente ambos códigos previamente dados. En el código de Tarjeta vemos que recibe {item, appeals, manejoAppeals, manejoNext} (va todo entre llaves porque recordemos que es un único objeto JS). Esas son las props de tarjeta. Su componente padre es App, que define y le envía los props al componente hijo.

¿Qué es Firebase? ¿Cuáles son sus características?

Es una plataforma de desarrollo de aplicaciones móviles y web de Google. Google proporciona toda la infraestructura del lado del servidor ya construida en la nube. (arquitectura BaaS: Backend-as-a-Service).

Entonces Firebase proporciona toda la infraestructura del lado del servidor, esto es bases de datos, seguridad, almacenamiento, servidores. Esto permite a los programadores enfocarse en el Frontend (por ejemplo usando React) sin tener que programar o controlar los componentes backend nombrados ni mantener servidores desde cero.

Características:

- Bases de datos noSQL en tiempo real (si un dato cambia en la base de datos, las pantallas de todos los usuarios conectados se actualizan automáticamente)
- Autenticación integrada. Permite implementar sistemas de inicio de sesión robustos escribiendo muy poco código.
- Hosting. Permite subir a la internet los archivos de la página web.
- Permite guardar y servir contenido generado por los usuarios, como fotos de perfil.